UNIVERSIDADE DE MOGI DAS CRUZES

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

8º SEMESTRE - SISTEMAS DE INFORMAÇÃO - NOTURNO
DISCIPLINA: APLICATIVOS WEB
PROF. ALESSANDRO APARECIDO DA SILVA HORAS

OTÁVIO RAMOS RIBEIRO

DASHBOARD ADMINISTRATIVO DE CHAMADOS TÉCNICOS

Mogi das Cruzes

2026

OTÁVIO RAMOS RIBEIRO

DASHBOARD ADMINISTRATIVO DE CHAMADOS TÉCNICOS

Trabalho desenvolvido para a disciplina de Aplicativos Web, do 8º semestre noturno do curso de Bacharelado em Sistemas de Informação da Universidade de Mogi das Cruzes, sob orientação do Prof. Alessandro Aparecido da Silva Horas, com foco no desenvolvimento de um Dashboard Administrativo integrado a uma API REST para acompanhamento de chamados técnicos.

Mogi das Cruzes

2026

Sumário

1. INTRODUÇÃO 2

1.1 OBJETIVO GERAL 2

1.2 OBJETIVOS ESPECÍFICOS 2

2. REQUISITOS DO SISTEMA 3

2.1 REQUISITOS FUNCIONAIS 3

2.2 REQUISITOS NÃO FUNCIONAIS 4

3. TECNOLOGIAS E FERRAMENTAS UTILIZADAS 5

4. ARQUITETURA E FUNCIONAMENTO DO SISTEMA 6

5. DOCUMENTAÇÃO DA API 6

5.1 PARÂMETROS DE FILTRO 7

5.2 EXEMPLOS DE RESPOSTA 7

5.3 TRATAMENTO DE ERROS 7

6. DASHBOARD ADMINISTRATIVO 8

7. TESTES E EVIDÊNCIAS 9

8. CONTROLE DE ACESSO, REPOSITÓRIO E VÍDEO 10

9. CONCLUSÃO 11

1. INTRODUÇÃO

O desenvolvimento de sistemas voltados ao acompanhamento de chamados técnicos exige que as informações sejam apresentadas de forma organizada e de fácil consulta. Quando os dados ficam disponíveis apenas em registros isolados, torna-se mais difícil acompanhar a quantidade de chamados, sua situação, os tipos mais recorrentes e o período em que foram concluídos.

Com base nessa necessidade, foi desenvolvido um Dashboard Administrativo integrado a uma API REST para consulta e acompanhamento de chamados técnicos. A solução foi incorporada ao projeto Login Seguro, que já possuía autenticação, autorização por perfis e gerenciamento de usuários, permitindo que o novo módulo aproveitasse a estrutura de segurança existente.

O Dashboard apresenta indicadores gerais, gráficos, filtros e uma tabela com informações dos chamados cadastrados. Os dados são armazenados no MongoDB Atlas e consultados pelo backend desenvolvido com Java e Spring Boot. A interface utiliza Thymeleaf, Bootstrap, JavaScript e Chart.js para apresentar as informações ao administrador.

1.1 OBJETIVO GERAL

Desenvolver um Dashboard Administrativo integrado a uma API REST para permitir o acompanhamento de chamados técnicos por meio de indicadores, gráficos, filtros e consulta das informações armazenadas no banco de dados.

1.2 OBJETIVOS ESPECÍFICOS

Disponibilizar indicadores gerais sobre os chamados técnicos cadastrados.
Apresentar a quantidade de chamados concluídos por período.
Permitir a visualização da situação dos chamados por meio de gráficos.
Identificar os tipos de chamados mais atendidos.
Disponibilizar filtros por data, status e tipo de chamado.
Integrar a interface do Dashboard aos endpoints da API REST.
Permitir a atualização dos dados exibidos sem necessidade de reiniciar a aplicação.
Restringir o acesso ao Dashboard e às rotas da API ao perfil de administrador.

2. REQUISITOS DO SISTEMA

Os requisitos foram organizados de acordo com as funcionalidades previstas para o Dashboard e com as características técnicas necessárias ao funcionamento da aplicação.

2.1 REQUISITOS FUNCIONAIS

RF01 - Indicadores gerais de chamados técnicos: O sistema deve apresentar o total de chamados e as quantidades de chamados abertos, em andamento, concluídos e cancelados.

RF02 - Análise de chamados atendidos por período: O sistema deve permitir a consulta dos chamados concluídos dentro de um intervalo de datas e apresentar os resultados de forma gráfica.

RF03 - Situação dos chamados: O sistema deve apresentar a distribuição dos chamados de acordo com sua situação atual.

RF04 - Tipos de chamados mais atendidos: O sistema deve apresentar a quantidade de chamados agrupados por tipo, permitindo identificar os tipos mais recorrentes.

RF05 - Filtros de consulta: O sistema deve permitir filtrar os chamados por período, status e tipo, atualizando os indicadores, gráficos e a tabela de resultados.

RF06 - Integração com a API: O Dashboard deve consumir os endpoints da API REST para obter os dados utilizados nos indicadores, gráficos, filtros e listagens.

RF07 - Atualização e tratamento de dados: O sistema deve permitir atualizar os dados do Dashboard e tratar falhas ocorridas durante as consultas à API.

2.2 REQUISITOS NÃO FUNCIONAIS

RNF01 - Segurança: As rotas administrativas e os endpoints relacionados ao Dashboard devem ser protegidos pelo Spring Security e acessíveis somente ao perfil ADMIN.

RNF02 - Persistência: Os dados dos chamados devem ser armazenados no MongoDB Atlas e acessados por meio do Spring Data MongoDB.

RNF03 - Usabilidade: A interface deve manter o padrão visual já utilizado pelo sistema, utilizando componentes simples e de fácil compreensão.

RNF04 - Manutenibilidade: O código deve permanecer organizado em camadas, separando controllers, services, repositories, models e DTOs.

RNF05 - Compatibilidade: A aplicação deve ser executável em navegador moderno e utilizar tecnologias compatíveis com o ambiente Java/Spring Boot adotado no projeto.

3. TECNOLOGIAS E FERRAMENTAS UTILIZADAS

O projeto utiliza tecnologias voltadas ao desenvolvimento web, à segurança, à persistência dos dados e à apresentação visual das informações do Dashboard.

Java 17: utilizado no desenvolvimento da aplicação e das regras de negócio.

Spring Boot: utilizado na organização do backend e no desenvolvimento da API REST.

Spring Security: responsável pela autenticação, autorização e proteção das rotas administrativas.

Spring Data MongoDB: utilizado para acesso e persistência dos dados no MongoDB Atlas.

Spring Session: utilizado no armazenamento das sessões autenticadas.

Thymeleaf: utilizado na construção das páginas HTML integradas ao backend.

Bootstrap 5 e Bootstrap Icons: utilizados na organização visual dos componentes e na interface responsiva.

JavaScript: utilizado para consumir a API e atualizar os elementos do Dashboard.

Chart.js: utilizado na construção dos gráficos apresentados ao administrador.

MongoDB Atlas: utilizado para armazenamento de usuários, sessões e chamados técnicos.

Git e GitHub: utilizados para controle de versão, organização das branches e disponibilização do projeto.

4. ARQUITETURA E FUNCIONAMENTO DO SISTEMA

A aplicação mantém uma organização em camadas. Os controllers recebem as requisições HTTP, os services concentram a lógica utilizada nas consultas e atualizações, e os repositories realizam o acesso ao MongoDB Atlas. Os models representam os dados persistidos, enquanto os DTOs são utilizados para organizar as informações retornadas ao Dashboard.

Na interface, o usuário administrador acessa o Dashboard pelo navegador. O JavaScript executado na página utiliza a função fetch para realizar requisições aos endpoints da API REST. O backend processa as consultas, acessa os dados armazenados e retorna as respostas em formato JSON. Em seguida, a interface atualiza os indicadores, gráficos e tabelas exibidos na tela.

Dashboard / Thymeleaf---JavaScript (fetch)--Controllers REST---Services---Repositories---MongoDB Atlas

O acesso ao Dashboard e aos endpoints relacionados aos chamados foi restrito ao perfil ADMIN. Dessa forma, usuários comuns ou moderadores não possuem autorização para consultar as informações administrativas diretamente pelas rotas protegidas.

5. DOCUMENTAÇÃO DA API

A API foi desenvolvida no próprio projeto Spring Boot e possui endpoints voltados à consulta, criação, filtragem e atualização dos chamados, além de rotas específicas para fornecer os dados apresentados pelo Dashboard.

Método	Endpoint	Finalidade
GET	/api/chamados	Listar todos os chamados cadastrados.
GET	/api/chamados/{id}	Buscar um chamado pelo identificador.
POST	/api/chamados	Cadastrar um novo chamado.
GET	/api/chamados/filtro	Filtrar chamados por status, tipo e período.
PUT	/api/chamados/{id}/status	Atualizar o status de um chamado.
PUT	/api/chamados/{id}/consideracao	Atualizar a consideração técnica.
GET	/api/dashboard/indicadores	Retornar os indicadores gerais do Dashboard.
GET	/api/dashboard/chamados-periodo	Retornar chamados concluídos por período.
GET	/api/dashboard/situacoes	Retornar a quantidade de

		chamados por situação.
GET	/api/dashboard/tipos	Retornar a quantidade de chamados por tipo.

5.1 PARÂMETROS DE FILTRO

O endpoint /api/chamados/filtro aceita parâmetros opcionais. O parâmetro status utiliza os valores ABERTO, EM_ANDAMENTO, CONCLUIDO ou CANCELADO. O parâmetro tipo utiliza os valores HARDWARE, SOFTWARE, REDE, ACESSO ou OUTROS. Os parâmetros dataInicio e dataFim utilizam o formato AAAA-MM-DD.

5.2 EXEMPLOS DE RESPOSTA

O endpoint de indicadores retorna um objeto JSON com as quantidades gerais utilizadas nos cards do Dashboard.

O endpoint de chamados concluídos por período retorna uma lista contendo a data e a quantidade correspondente.

5.3 TRATAMENTO DE ERROS

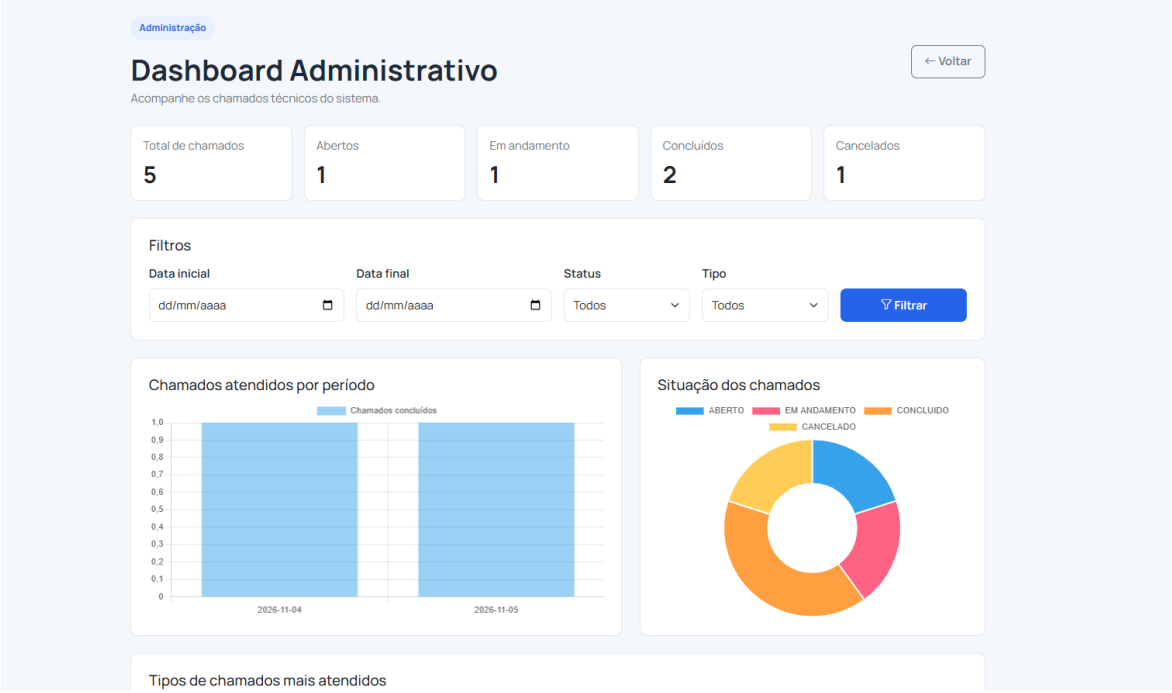
Quando um chamado solicitado por identificador não é encontrado, o service gera uma exceção tratada pelo controller, retornando uma resposta de erro ao cliente. No Dashboard, as requisições também são verificadas antes da leitura do JSON. Caso uma chamada à API não seja concluída corretamente, a interface informa que não foi possível carregar ou atualizar os dados.

6. DASHBOARD ADMINISTRATIVO

O Dashboard Administrativo foi construído para centralizar as principais informações dos chamados técnicos em uma única tela. O acesso ocorre por meio da Área Administrativa e é permitido somente a usuários autenticados com o perfil ADMIN.

Na parte superior são apresentados os indicadores gerais: total de chamados, chamados abertos, em andamento, concluídos e cancelados. Abaixo dos indicadores estão disponíveis os filtros por data, status e tipo. Os gráficos apresentam os chamados concluídos por período, a distribuição por situação e os tipos mais atendidos.

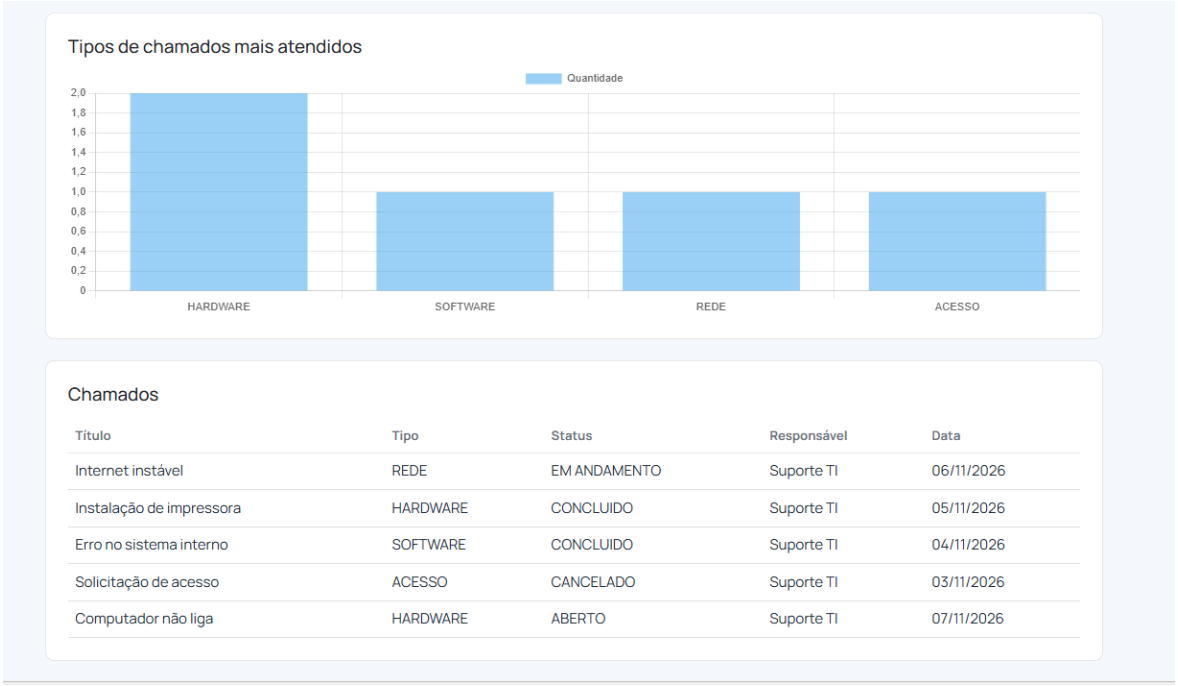
Figura 1 - Visão geral do Dashboard Administrativo



Fonte: Elaborado pelo autor (2026).

O gráfico de período utiliza os dados de conclusão dos chamados, enquanto o gráfico de situação demonstra a distribuição entre chamados abertos, em andamento, concluídos e cancelados. O gráfico de tipos permite identificar quais categorias apresentaram maior quantidade de registros.

Figura 2 - Tipos de chamados e listagem de registros



Fonte: Elaborado pelo autor (2026).

A tabela apresenta as informações dos chamados retornados pela API. Na versão final da atividade, a listagem também apresenta a consideração técnica registrada para cada chamado. Os filtros atualizam não apenas a tabela, mas também os indicadores e os gráficos de acordo com o conjunto de dados encontrado.

O botão Atualizar dados executa uma nova consulta aos endpoints da API. Antes de reconstruir os gráficos, as instâncias existentes são removidas, evitando duplicações durante a atualização da página.

7. TESTES E EVIDÊNCIAS

Os endpoints foram testados durante o desenvolvimento por meio do navegador, utilizando uma sessão autenticada com o perfil de administrador. Os resultados retornados pela API foram comparados com os registros cadastrados na collection chamados do MongoDB Atlas.

O filtro por tipo HARDWARE retornou os dois chamados cadastrados nessa categoria, demonstrando o funcionamento da consulta por parâmetro. Também foram realizados testes com filtros por status e por período, além da validação dos endpoints utilizados pelos gráficos.

Figura 3 - Teste do filtro de chamados por tipo HARDWARE



Fonte: Elaborado pelo autor (2026).

No teste dos indicadores gerais, a API retornou cinco chamados no total, sendo um aberto, um em andamento, dois concluídos e um cancelado. O endpoint de período retornou um chamado concluído em 4 de novembro de 2026 e um chamado concluído em 5 de novembro de 2026. Os dados apresentados no Dashboard corresponderam aos resultados obtidos nos testes da API.

8. CONTROLE DE ACESSO, REPOSITÓRIO E VÍDEO

O sistema utiliza o Spring Security para controlar o acesso às funcionalidades disponíveis. As rotas relacionadas ao Dashboard e à API de chamados foram configuradas para permitir o acesso somente a usuários com perfil de administrador. Dessa forma, o Dashboard permanece integrado ao mecanismo de autenticação e autorização já existente no projeto Login Seguro.

Para a avaliação do projeto, foi disponibilizada uma conta administrativa específica, permitindo que o professor realize o login e acesse diretamente o Dashboard e as demais funcionalidades destinadas ao perfil de administrador.

O código-fonte do projeto, o vídeo de apresentação e as informações necessárias para execução e avaliação da aplicação foram disponibilizados no repositório público do projeto no GitHub.

O desenvolvimento utilizou o fluxo de branches feature/dashboard-chamados, develop e main. Após a conclusão dos testes, a branch de funcionalidade foi integrada à develop e, posteriormente, à main, mantendo o histórico das alterações realizadas.

9. CONCLUSÃO

O desenvolvimento da atividade permitiu integrar ao projeto existente um Dashboard Administrativo voltado ao acompanhamento de chamados técnicos. A solução passou a disponibilizar indicadores gerais, gráficos, filtros e listagem de chamados, utilizando os dados persistidos no MongoDB Atlas.

A comunicação entre a interface e o backend foi realizada por meio de uma API REST desenvolvida com Spring Boot. A utilização de JavaScript e Chart.js permitiu apresentar os dados retornados pela API de forma visual, enquanto o Spring Security foi mantido como mecanismo de proteção das rotas administrativas.

Os requisitos previstos para o Dashboard foram implementados e testados. O sistema permite ao administrador consultar os principais dados dos chamados, aplicar filtros e atualizar as informações apresentadas, mantendo a organização em camadas e a integração com as funcionalidades já existentes no projeto Login Seguro.